

From FiGS to Semantic Goals:

A Chronological Account of the Implementation, the Problems Met and How They Were Solved

Suhan — Team Radiance, Department of Computer Science and Engineering,
University of Moratuwa

8 October 2026

Abstract

This document records, in the order it happened, everything we implemented between July and October 2026: validating and installing Stanford MSL’s FiGS / SOUS-VIDE stack on our own hardware, a resumable capture-to-flight pipeline, the Galley web console (job queue, splat page, course editor, SV-Net pages, splat editor), and the semantic layer that turns a typed phrase into a 3D goal on a frozen Gaussian splat (teacher features, the lift, FMGS, the query, the evaluation and four teacher designs). For each step it gives what was done, the problems met — version drift, host faults, silent data errors, GPU limits, power cuts, our own annotation mistakes — and how each was diagnosed and fixed, with the measured result.

Contents

1	Machines	3
2	Getting FiGS to run (25 Jul – 7 Aug)	4
2.1	Feasibility check on the 4 GB laptop (25 Jul)	4
2.2	A clean rebuild with an install script (2 Aug)	4
2.3	A portable install on an SSD	5
2.4	First complete capture-to-flight on the lab machine (6 Aug, intellisense05)	5
2.5	One resumable pipeline script (7 Aug)	6
3	Galley, the web console (27 Sep – 4 Oct)	6
3.1	Galley phases 0–3 (built in the cloud, gated on dummy, committed 27 Sep)	7
3.2	SV-Net through Galley (27 Sep – 1 Oct)	8
3.3	Console features and the Windows 7 ribbon redesign (1 – 4 Oct)	9
4	Semantic goals (5 – 8 Oct)	10
4.1	Phase 0 — environment and pose check (5 Oct)	10
4.2	Phase 1 — teachers, lift and query (5 Oct)	11
4.3	Phase 2 — Galley backend and query worker (5 Oct)	11
4.4	Phase 3 — the splat editor (5 Oct)	11
4.5	Phase 4 — FMGS on a frozen splat (5 – 6 Oct)	12
4.6	Phase 5 — evaluation on two scenes (6 Oct)	12
4.7	Why the misses? Diagnostic and ground-truth check (7 Oct)	13

4.8	Teacher variants A, B, C (7 – 8 Oct)	14
4.9	Improving B and C, and a floor sweep (8 Oct)	14
5	Lessons	15
6	Timeline	16

Team Radiance (Suhan), CSE, University of Moratuwa. Written 8 Oct 2026 from the repository history (73 commits, 7 Aug – 8 Oct 2026), the run records and the project docs (FiGS_validation_notes.md, LAB_MACHINE.md, FiGS_pipeline_script_guide.md, README_PORTABLE.md, GALLEY_UI.md, SEMANTICS.md). Every step is listed in the order it happened, with the problems met along the way and how each was solved. The Markdown source is docs/PROJECT_CHRONICLE.md.

The project: **language-guided goals for simulation-trained visuomotor drone policies**. Stanford MSL’s SOUS-VIDE turns a phone video of a room into a Gaussian-splat simulator (FiGS), flies an expert controller through it and distils the flights into a visuomotor student policy (SV-Net). We added an install and pipeline layer around it, a web console (Galley), and a semantic layer that turns a typed phrase into a 3D goal.

1 Machines

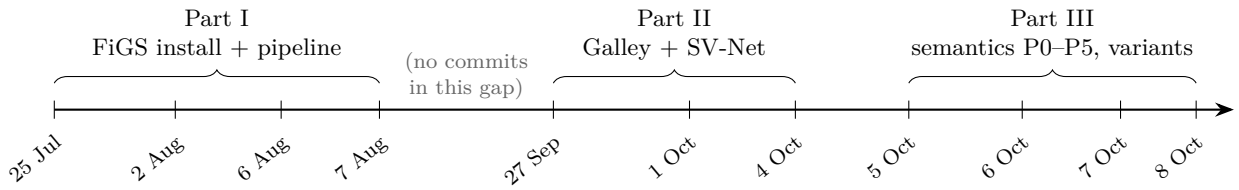


Figure 1: Timeline of the work (not to scale). Laptop and SSD in July–August, intellisense05 in August, intellisense08 from late September.

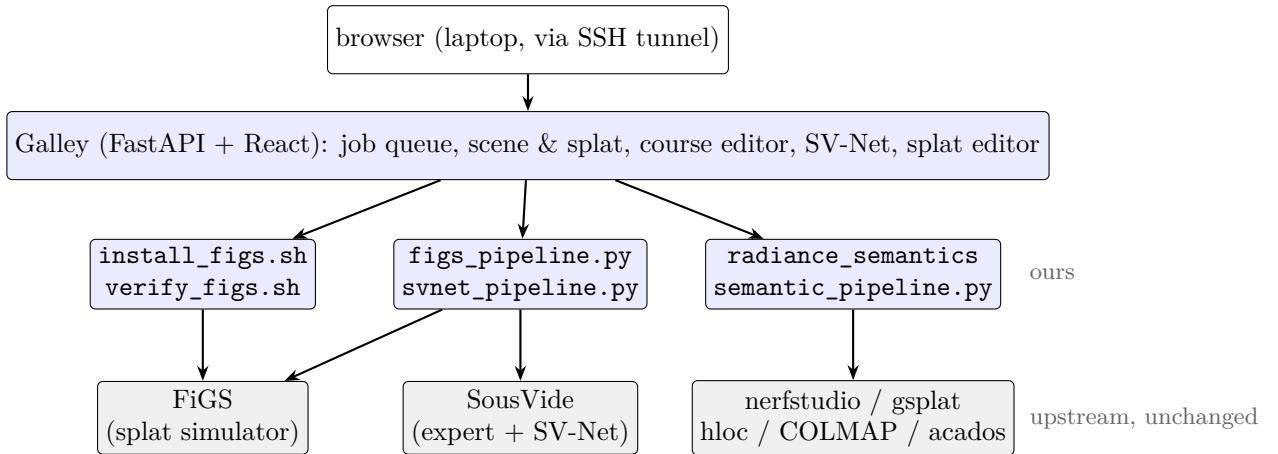


Figure 2: What was built around the upstream code. Grey: upstream (pinned, unchanged); blue: this project.

Machine	GPU	Role over time
dummy (home laptop, Ubuntu 24.04)	RTX 3050 Ti Laptop, 4 GB	July–Aug feasibility and first install; later the Galley build-and-test host
Portable SSD install	(host’s GPU)	The whole environment on an SSD, mounted on other Ubuntu machines
intellisense05 (lab, campus-only)	RTX 2080, 8 GB	First complete capture-to-flight run (6 Aug)
intellisense08 (lab, Tailscale)	RTX 2080, 8 GB, 62 GB RAM, Ubuntu 22.04	From 28 Sep: SV-Net host, then every semantics gate and experiment
RTX 5060 Ti PC (home)	16 GB, Blackwell	Future main host (needs CUDA 12.8 / torch ≥ 2.7)

2 Getting FiGS to run (25 Jul – 7 Aug)

2.1 Feasibility check on the 4 GB laptop (25 Jul)

What we did. Read the SOUS-VIDE paper and the three upstream repos (FiGS, FiGS-Examples, SousVide). Checked the laptop: RTX 3050 Ti 4 GB, driver 570 (CUDA 12.8), 16 GB RAM, Ubuntu 24.04. The upstream environment needs Python 3.10, torch 2.1.2 with its own conda CUDA 11.8 toolkit, tiny-cuda-nn built from source, nerfstudio 1.1.4 (with gsplat), COLMAP, acados (built with cmake) and hloc.

Main risk identified. Splat training (splatfacto) was thought to need about 6 GB of VRAM, more than the card had. Plan: use the laptop for simulation and test training later at reduced settings.

Problems and fixes.

Problem	Cause	Fix
conda not installed	fresh machine	installed Miniconda
tiny-cuda-nn build: pkg_resources missing	pip's build isolation pulled a new setuptools	<code>pip install "setuptools<81" and --no-build-isolation</code>
tiny-cuda-nn build used the wrong compiler	system nvcc 12.8 ahead of conda's CUDA 11.8 on PATH; then CUDA 11.8 rejected Ubuntu 24.04's GCC 13	put <code>\$CONDA_PREFIX/bin</code> first, <code>CUDA_HOME=\$CONDA_PREFIX</code> , install <code>gcc-11/g++-11</code> and use them as <code>CC/CXX</code>
example splats would not download	Google Drive's per-file quota on the shared academic link	downloaded in the browser, copied with <code>scp</code> (5 GB: backroom, flightroom, mid_gate, src_open)
example notebook asked for a missing scene	default <code>button</code> scene not in the download	pointed it at <code>backroom + circuit</code> (the paper's cluttered trajectory)

Result. The example flight (backroom + circuit) ran headless with 0 failing cells and produced a rendered video; VRAM returned to idle. Simulation fits easily in 4 GB.

2.2 A clean rebuild with an install script (2 Aug)

What we did. Wrote `setup_scripts/install_figs.sh` (step-addressable, resumable, logs per step, `--redo <step>`) and `verify_figs.sh`, and rebuilt the environment with them. Verification: 22 passed, 0 failed. Measured: backroom/circuit flight 40 s, peak 608 MiB; tiny-cuda-nn build 2 min 29 s for one architecture.

Problems and fixes — four host faults that cost most of a day.

Problem	Cause	Fix
nvidia-smi: cannot communicate with the driver	an unattended upgrade installed kernel 7.0; the 570 driver's DKMS module builds only for 6.17. GRUB_DEFAULT=saved + GRUB_SAVEDefault=true is not a pin: one boot into 7.0 made it sticky	pin GRUB to an explicit menuentry ID of 6.17.0-35, GRUB_SAVEDefault=false, grub-editenv - unset saved_entry
driver installed but Key was rejected by service	Secure Boot with the DKMS signing key not enrolled	mokutil --import and enrolment at the MOK screen on reboot (needs physical access)
install truncated mid-way	the desktop suspended the machine when idle	masked sleep/suspend/hibernate targets; lid and idle actions ignored
machine hard-locked during a JIT compile	ninja ran one nvcc per thread: 20 threads $\times$ 1-2 GB > 16 GB RAM + swap	MAX_JOBS=4 baked into figs_env.sh; recovery needed Magic SysRq
two versions of nerfstudio and gsplat at once; gsplat compiled an orphaned sh.cu (DEVICE_GUARD undefined)	the interrupted pip install left both dist-info directories	hard purge of both, reinstall; a duplicate-distribution check added
preflight error NVIDIA: unbound variable	the script tested that nvidia-smi exists, not that it runs, and fed its error text into arithmetic	detect "installed but not communicating" and print the remedy

Pinned versions (drift was observed and corrected): nerfstudio 1.1.4 (1.1.5 pulls an incompatible gsplat), gsplat 1.0.0, torch 2.1.2 / torchvision 0.16.2 (every compiled extension is built against this ABI), numpy 1.26.4 (numpy 2 breaks the C ABI).

2.3 A portable install on an SSD

What we did. Put the whole environment (conda, the **kitchen** env with its CUDA 11.8, SousVide, acadoss, splats) on an external SSD so other lab machines could use it (**README_PORTABLE.md**).

Problems and fixes. Conda environments are not relocatable (absolute paths in shebangs and .pth files), and Ubuntu mounts drives under /media/<user>/<UUID>, so the path changes with the user $\rightarrow$ mount explicitly at the original path by UUID; the installer detects a mismatch. The NVIDIA driver and apt packages belong to the host; tiny-cuda-nn is compiled for one GPU architecture and must be rebuilt for another (15-40 min); the environment needs the same or a newer Ubuntu (glibc).

2.4 First complete capture-to-flight on the lab machine (6 Aug, intellisense05)

What we did. Installed in place on intellisense05 (RTX 2080 8 GB, TCNN_CUDA_ARCHITECTURES=75) and ran our own capture end to end: a Samsung S23 4K60 HEVC video (191.7 s, 3.0 GB) with an 18 cm ArUco marker, transcoded to 1080p30, 300 frames, hloc SuperPoint + SuperGlue, COLMAP, ArUco metric alignment, splatfacto 30k steps, then a simulated flight.

Result	Value
Registration	300 / 300 images, 40,542 points, reprojection error 1.43 px
SuperGlue matching	38 min (44,850 exhaustive pairs)
Splat training	34 min, peak 5,203 MiB
Flight simulation	51 s for 16 s of flight, peak 635 MiB
Whole pipeline	about 1 h 40 min

Problems and fixes.

Problem	Cause	Fix
install and verify scripts said a healthy install was missing	default prefix <code>~/projects/figs_validation</code> hard-coded	<code>--prefix</code> every time; later scripts resolve the project root themselves
<code>import_images()</code> : incompatible function arguments	pycolmap 4.1.1 renamed <code>image_list</code> to <code>image_names</code> ; hloc still used the old name	one-line patch in the hloc submodule (later applied automatically by the pipeline’s patch step)
DataLoader worker ... Segmentation fault at 0/300	OpenCV 5.0.0 inside forked DataLoader workers	<code>num_workers=0</code> in hloc’s feature extraction (patched automatically)
<code>libqpOASES_e.so</code> : cannot open shared object file in <code>Simulator()</code> (docs/errors.txt)	the environment was activated with <code>conda activate</code> only; <code>LD_LIBRARY_PATH</code> and <code>ACADOS_SOURCE_DIR</code> come from <code>figs_env.sh</code>	always <code>source figs_env.sh</code> ; preflight now loads the library by soname in the first seconds
drone flew “below the floor”, video looked like mush	course files are z-down and y-negated relative to <code>transforms.json</code> : <code>course = (x, -y, -z)</code>	derived and documented the conversion; every waypoint is validated against the captured volume
<code>ns-viewer</code> held ~4.6 GB CasADi 3.7.2 newer than acados supports	left running unpinned	close it before training works; noted as first suspect if codegen breaks
marker detections clustered in time	most marker frames in the first 20 s	<code>num_marked</code> raised from 20 to 40 so the late window contributes

2.5 One resumable pipeline script (7 Aug)

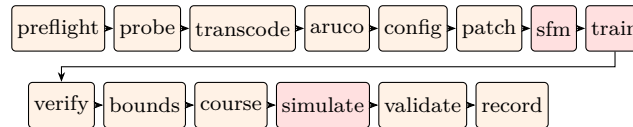


Figure 3: The pipeline’s steps (the `gsplat` step was split into `sfm` + `train` on 27 Sep). Red: GPU steps. Each step leaves a marker with a fingerprint of its inputs, so a re-run resumes after the last finished step.

What we did. Replaced the notebook with `figs/figs_pipeline.py`: steps `preflight` → `probe` → `transcode` → `aruco` → `config` → `patch` → `gsplat` → `verify` → `bounds` → `course` → `simulate` → `validate` → `record`, state on disk (an interrupt costs one step), VRAM sampling, an ArUco time histogram, and a table of known failure modes printed with their remedy. First commit of this repository (35efbad): “validated end to end on an RTX 2080: 300/300 frames registered, 5,203 MiB peak training VRAM, ~1 h 40 m”.

Quiet failures the script exists to catch: a wrong marker length (a perfectly reconstructed room at the wrong scale), a wrong marker ID (alignment fails an hour later), partial registration, waypoints outside the captured volume.

3 Galley, the web console (27 Sep – 4 Oct)

Galley is a FastAPI + React console in `ui/` that drives the pipeline scripts. Design rule: the scripts stay the source of truth; Galley only chooses flags and step ranges and reads the scripts’ state files,

so anything started in the browser can be resumed from the shell. Work in this period was written by Claude in a cloud sandbox; Suhan committed, pulled on the hosts, ran gate scripts and pasted the logs back.

3.1 Galley phases 0–3 (built in the cloud, gated on dummy, committed 27 Sep)

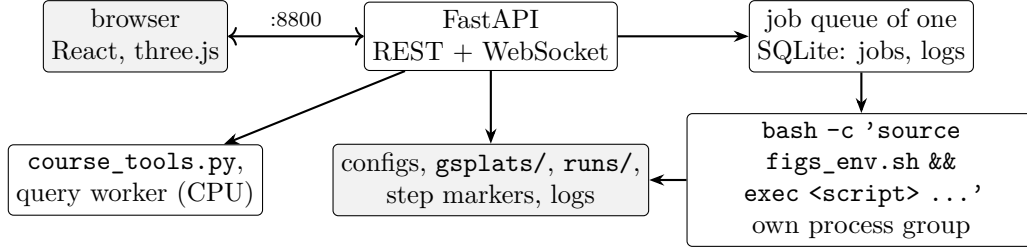


Figure 4: Galley’s architecture. The scripts remain the source of truth: Galley builds validated argument lists, runs one GPU job at a time, and reads the scripts’ state files.

Phase	What it added	Gate result
0	dummy aligned with the lab setup	backroom + circuit flown by <code>figs_pipeline.py</code> : 34 s, tracking max 0.034 m, 614 MiB
1	backend core: SQLite job queue of one, config models, log streaming over WebSocket, cancel by process group	26 tests, a real preflight job through the queue, cancel of a running job
2	Scene & Splat page; the pipeline’s <code>gsplat</code> step split into <code>sfm</code> + <code>train</code>	retrain from the UI, training curve shown, previous model promoted back, flight re-run
3	course editor: 3D waypoints over the SfM points, MinTimeSnap preview, clearance, Save and fly	a browser-built course passes <code>course</code> and flies (tracking max 0.069 m)

Problems and fixes.

Problem	Cause	Fix
the known-good backroom + circuit was refused by the <code>course</code> step	a <code>null</code> (free) waypoint cell became NaN	check only constrained axes
<code>simulate</code> failed writing the video	imageio’s FFMPEG writer refuses a <code>.partial</code> extension (ffmpeg likewise in <code>transcode</code>)	write <code><scene>_flight.partial.mp4</code> , rename; <code>-f mp4</code>
a retrain repeated SfM (~45 min)	one <code>gsplat</code> step	split into <code>sfm</code> (upstream <code>generate_gsplat</code> with its <code>ns-train</code> call intercepted) and <code>train</code>
preflight warned “under 6 GB, training will OOM”	wrong assumption	measured: with <code>--cache-images cpu</code> , backroom trains on the 4 GB laptop at 2,360 MiB (60 min); most of the 5.2 GB reference was images cached on the GPU
editing a course under the same name kept the old flight	step fingerprints did not include the course file	fingerprints include a course digest
a course saved from the browser flew a different path	browsers write 0.0 as 0; FiGS’s <code>KF_to_TpF0</code> reads an integer cell as the previous cell’s value, silently	Galley writes floats; the <code>course</code> step refuses integer cells; a lint endpoint finds them
clearance put the known-good circuit 2.7 cm from an obstacle	isolated SfM outlier points	clearance = distance to the 5th-nearest sparse point
stretching keyframe times did not slow the flight	the expert’s <code>kT</code> makes <code>MinTimeSnap</code> re-optimize durations; file times are only a starting guess	<i>Re-time like expert</i> preview and <i>Write solved times</i>

3.2 SV-Net through Galley (27 Sep – 1 Oct)

What we did. `figs/svnet_pipeline.py` reproduces the upstream SOUS-VIDE notebook step for step (`preflight` → `rollout` → `observe` → `train_hist` → `train_comm` → `deploy`), calling upstream code unchanged, resumable per cohort, plus SV-Net pages in Galley.

Problems and fixes.

Problem	Cause	Fix
commNet out of memory in epoch 2 on the 4 GB laptop	every step in one process: the splat, observation tensors and PyTorch's cache stayed on the card	one child process per step; <code>gc.collect()</code> + <code>empty_cache()</code> after deploy; test pass without autograd
no progress in logs	rich progress bars draw nothing when stdout is a pipe	plain progress lines and per-epoch losses in <code>live_*.jsonl</code>
a smaller re-run mixed old and new rollouts	upstream overwrites by index	old data archived before a re-run
a second <code>train_hist</code> continued the first	<code>Pilot()</code> loads the network on disk	<code>--fresh histNet</code> or <code>--fresh commNet</code>
upstream's tracking error looked fine for bad flights	<code>compute_flight_metrics</code> uses <code>axis=0</code> (the whole path per axis), not the distance to each point	our per-point tracking error reported next to upstream's
commNet needs ~6.8 GB with in-loop evaluation	training + the splat for evaluation flights	SV-Net moved to <code>intellisense08</code> (8 GB)

Lab bring-up (28 Sep – 1 Oct). `ui/deploy/host.sh` (paths by hostname), `bringup.sh`, a machine profile `ui/machines/intellisense08.toml`; the repository moved to the Radiance-ICE-22 organisation (**Radiance-Environment-Setup**). verify 21/21; backroom + circuit identical to dummy (tracking max 0.034 m).

Gate result (1 Oct, 49 min, cohort p4_smoke). Every step ran through the queue; commNet peaked at 6,785 MiB. The expert tracks the course to 1.7 cm; **the student (Maverick) does not fly it** (mean 9.4 m off, 16 % within 0.3 m). Next for SV-Net: watch its video, then a larger dataset (`data_beta`). Still open.

3.3 Console features and the Windows 7 ribbon redesign (1 – 4 Oct)

- **Drone model (1 Oct):** the team's CAD (1.62 M triangles) converted to a 159k-triangle GLB in FiGS's body frame; measured bounding sphere 0.19 m; clearance now reports the *gap* (centre distance minus 0.19 m), default minimum 0.15 m.
- **run_ui.sh (1 Oct):** start/stop/status per host; refuses to stop while a job runs (a stopped server orphans the job).
- **Ribbon redesign (1 Oct):** designed in Figma (design system, ribbons, five screens), then built into the frontend: ribbon with hover help for every command, Explorer, tabbed documents, Properties, Output, status bar, shortcuts. Problems: Figma's Starter plan allows 3 pages and 20 MCP calls a month → screens generated by a local Figma plugin; Noto Sans lacks →, ≤ and box glyphs → reworded.
- **3–4 Oct:** full screen, resizable tiles; a stale `index.html` after deploys → served with no-cache, plus a “newer build” notice and an end to an endless “Loading the 3D editor”.
- **Splat in the course editor (4 Oct):** the checkpoint exported to the 32-byte `.splat` format (cached, keyed by run + checkpoint + mtime); drei's loader flips (x, y, z) → (x, -y, -z), which is exactly splat → course. Arrow-key flying.
- **Video upload and Google Drive import (4 Oct):** resumable chunked uploads; from home the upload crawled at ~0.5 MB/s (Tailscale relaying through DERP Bangalore) → the host downloads picked files directly from Google Drive (OAuth `drive.file`, token in memory, Range resume, MD5 check).
- **A log with 90k lines (4 Oct):** `figs_pipeline.sh()` read child output in text mode, turning every `tqdm \r` redraw into a new line (hloc's matching stored ~90k lines and looked stuck) →

output passed as bytes; Galley updates a redrawn line in place.

Workflow traps of this period: git from the Cowork VM left lock files on the laptop repo; the laptop copy is CRLF; dummy diverged once because files were copied with `scp` ($\rightarrow$ `pull.ff` only, update only by pulling); `--train-arg` needs the `=` form.

4 Semantic goals (5 – 8 Oct)

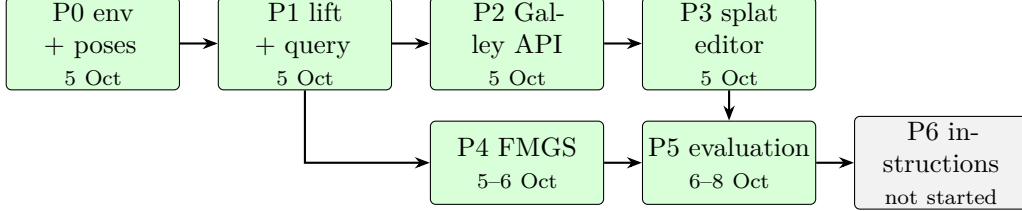


Figure 5: The semantics phases and their gates (green: passed).

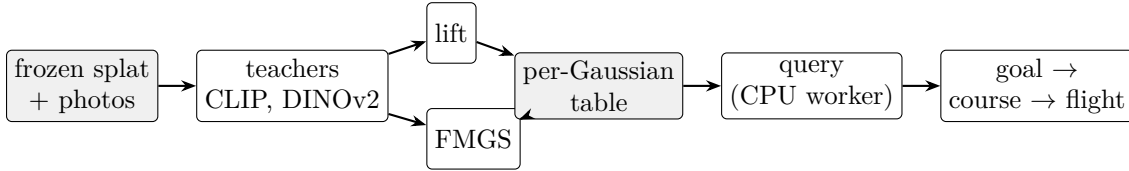


Figure 6: The semantic layer in one line (details in `semantic_embedding_report.pdf`).

The plan (`SEMANTICS_PLAN.md`) has seven phases, each closed by a gate on `intellisense08`: P0 environment and poses, P1 the training-free lift and a CLI query, P2 Galley’s API and a CPU query worker, P3 a splat editor (query $\rightarrow$ course $\rightarrow$ flight), P4 the FMGS backend, P5 the evaluation, P6 instructions $\rightarrow$ courses $\rightarrow$ SV-Net.

4.1 Phase 0 — environment and pose check (5 Oct)

What we did. Package `radiance_semantics`; `install_semantics.sh` adds OpenCLIP and DINOv2 to `kitchen` without moving `torch`; probes for `gsplat` feature rendering; a camera check.

Problems and fixes.

Problem	Cause	Fix
newest OpenCLIP needs <code>timm>=1.0.17</code> ; <code>nerfstudio</code> pins <code>timm==0.6.7</code>	version conflict	OpenCLIP pinned at 2.24.0 (<code>timm</code> optional), <code>huggingface_hub<1</code>
a constraints file from <code>pip freeze</code> constrained nothing	conda packages appear as <code>name @ file://...</code> ; in the cloud test env <code>numpy 2.2</code> slipped in and <code>torch</code> broke (<code>_ARRAY_API</code> not found)	constraints built from package metadata; every install <code>--no-deps</code> + constraints, <code>torch</code> re-checked
first gate: Unsupported number of channels: 64 in <code>rasterize_to_pixels_bwd</code>	<code>gsplat 1.0.0</code> ’s backward kernel takes ≤ 32 channels (forward ≤ 512)	<code>render_features</code> splits grad-requiring features into 32-channel chunks (29 passes per image for the lift)
features would be projected through wrong cameras	<code>splatfacto</code> applies its learned pose correction only while training	<code>cameras.optimized_c2w</code> applies it explicitly: +4.22 dB PSNR on backroom (31.68 vs 27.46 dB), corrections up to 39 mm / 2°

Second gate (09:48): all five checks passed.

4.2 Phase 1 — teachers, lift and query (5 Oct)

What we did. `semantic_pipeline.py` (resumable steps `preflight` → `cameras` → `teachers` → `lift` → `export`); CLIP crop pyramid (7 scales, averaged) and DINOv2 maps per photo; the blend-weighted lift onto 532,361 Gaussians; the table; `semantic_query.py` with LERF relevancy and voxel clusters; five annotated development queries.

First gate (fc7aed2): teachers 31.5 min, lift 3 min 11 s (7.6 GB), 3 of 5 hits.

Problem	Cause	Fix
red tool chest missed by 1.3 m	our annotation was placed at the wrong depth	Suhan re-placed it from the splat
whiteboard: 75,596 Gaussians selected, 10 m clusters	a fixed 0.55 floor lets every white wall and ceiling in	relative threshold $\tau = 0.55 + 0.5 \cdot (\text{peak} - 0.55)$, peak = mean of the top 100
lift peak 7.6 GB	the full 512-channel CLIP map upsampled at once	upsample per 32-channel chunk
10,741 “seen” Gaussians in the lift vs 11,592 in the table	weights in (0, 1e-8]	unseen means weight exactly 0, everywhere

Second gate (16:12): **5 of 5**, mean error 0.23 m. The five queries had shaped the threshold, so they became a development set; the evaluation would need a separate frozen set.

4.3 Phase 2 — Galley backend and query worker (5 Oct)

What we did. A long-lived CPU-only query worker (`semantic_worker.py`, JSON lines; keeps the CLIP text tower and memory-mapped tables), Galley’s client and the semantics API. Stale tables are detected by the same key as Galley’s `.splat` cache.

Problem	Cause	Fix
one backend test failed only on the host (405 instead of 404)	httpx normalised <code>/api/scenes/./semantics/query</code> ; with a built frontend present the path fell through to the static mount	the test uses a scene name the route rejects (400)

Gate: cold query 5.8 s, warm 716–789 ms, 5/5 hits — identical to the CLI.

4.4 Phase 3 — the splat editor (5 Oct)

What we did. `#/splat/<scene>`: the splat rendered in the browser (renderer vendored from drei with a recolour path), relevancy heatmap, candidates, picking, annotations, and **Send to course** (the approach point becomes the final keyframe, yaw facing the object).

Problem	Cause	Fix
a lone candidate showed “runner-up at 100 %”	the UI misread the margin (top – runner-up)/top	text fixed
a contextual ribbon tab reset to Home	a lazily loaded document registered after the reset	ribbon fix

Gate: “red tool chest” → course → flight (tracking max 8 mm); browser half: the whiteboard sent from the editor and flown (tracking max 73 mm); recolouring 532k Gaussians in 20 ms + 71 ms.

4.5 Phase 4 — FMGS on a frozen splat (5 – 6 Oct)

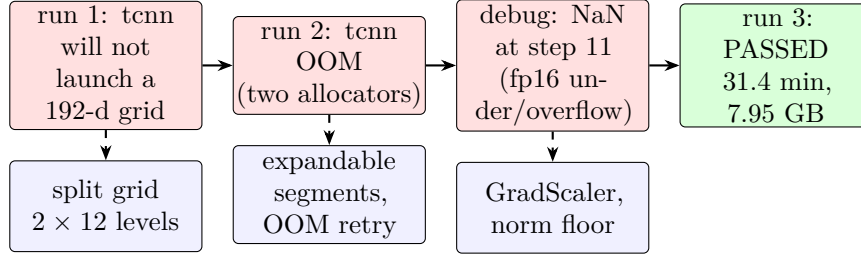


Figure 7: The Phase 4 gate: three failures, each with its fix, before the pass.

What we did. A standalone FMGS trainer (decided over an `ns-train` plugin: the splat is frozen, so only cameras and teacher maps are needed, and it shares the lift’s cameras and renderer for a fair comparison): hash-grid feature field, CLIP and DINO heads, CLIP Huber + DINO L2 + pixel-alignment loss, 4,200 steps; bake to the same table; View > Compare in the editor.

Gate run	Problem	Cause	Fix
1 (73e7968)	tiny-cuda-nn invalid argument	on sm_75 any 192-dim hash grid fails to launch (every grid ≤ 96 dims runs) — a configuration width limit, not memory	<code>fmgs/diag.py</code> probes; the 24 levels built as two 12-level grids (16→84, 98→512; every level within 1 %); per-part PyTorch fallback
2 (70d2178)	<code>cuMemCreate ... CUDA_ERROR_OUT_OF_MEMORY</code> in tcnn’s backward	live tensors peaked at 5.9 GB, but PyTorch’s cache kept 1–3 GB of fragmented free blocks that tcnn’s own allocator cannot use; the OOM ladder caught only <code>torch.cuda.OutOfMemoryError</code>	expandable segments; tcnn’s <code>RuntimeError</code> recognised; a failed step retried once; shorter tensor lifetimes
(debug)	loss NaN at step 11	CLIP gradients ($\sim 1e-8$) underflowed in tcnn’s fp16 before its $\times 128$ loss scale; pixel alignment normalised ~ 0 vectors, gradients $2.25e4$ overflowed	dynamic loss scaling (GradScaler); $1e-3$ norm floor in pixel alignment
3 (7344e7a)	—	—	PASSED 6 Oct 13:31: 4,200 steps in 31.4 min, peak 5.5 GB PyTorch / 7.95 GB device, Gaussians and checkpoint unchanged; dev queries FMGS 4/5, lift 5/5

Hand-off to Claude Code on the host (6 Oct). From here Claude ran directly on `intellisense08` (tests, probes, gates, logs), guided by `CLAUDE.md`. Gate run 2’s diagnosis and the fixes above were the first work done that way. `pytest` was added to `kitchen` with `--no-deps` and the frozen constraints; 68 semantics tests pass.

4.6 Phase 5 — evaluation on two scenes (6 Oct)

What we did. Four variants — L-C (lift, CLIP query), L-CD (lift + DINO at query time: diffusion over a DINO-weighted kNN graph and a DINO split of merged candidates), F-C (FMGS on CLIP alone), F-CD (FMGS with its paper losses) — on two scenes; an evaluator (multi-instance hits, failure kinds, absent-object false positives and AUROC); query sets annotated from photos and frozen before any result (f38225b: backroom 13 + 3 absent, flightroom 16 + 3).

Problem	Cause	Fix
GTN_lab_v1 looked non-metric (camera path 1,309 m, bounds ± 168 m)	scale was right (marker check: median ratio 1.003); 3 of 600 cameras registered 100–219 m away	<code>pose_inliers</code> drops cameras $> 4 \times$ the p90 distance from the median
GTN_lab_v1's room is folded	SfM fused two walls with the same logo (a “doppelganger” failure); 395 of 600 cameras face one direction; HDR footage without tone mapping (18 dB PSNR)	second scene changed to flightroom (mocap poses); GTN workspace deleted at Suhan's request, video kept
intellisense_v1 failed in SfM alignment	no ArUco marker visible in the footage	capture deleted; re-shoot with the marker needed
annotations from sparse SfM points leaked to the background on thin objects	few points on monitors, tripods, chairs	<code>locate --depth</code> : picks from the splat's rendered depth
host reset twice mid-training (17:47, ~18:14)	lightning power cuts	FMGS checkpoints every 1,000 steps; jobs resumed from step 3,000; GPU telemetry logging added for the diagnosis and removed afterwards at Suhan's request
evaluation reported the wrong build time for resumed runs	only the last leg was timed	build time = steps $\div$ it/s
the expert flew a semantic course at 33 m/s, 528 % thrust	the editor's <i>Add</i> halved a leg's time; MinTimeSnap (SLSQP on raw durations) stalls at a badly squeezed guess and reports success	<i>Add</i> inserts time (distance $\div$ mean speed); fast legs flagged in the editor, lint and the course step; a stalled re-time is reported
the editor scored only one instance and no absent objects	written for the dev set	multi-instance and negative scoring like the evaluator
the frontend could not be rebuilt on the host	no Node.js, no sudo	Node 22.23.3 under ~/Radiance/tools

Results (frozen sets). Lift 8/13 and 10/16 (≈ 0.62), ~ 0.2 m median error; F-C 6/13, 8/16; F-CD 7/13, 4/16 — F-CD's CLIP channel fragmented into tiny clusters (DINO dominates its loss). The relevancy floor is the only query setting that moves hits; the lift's render width (480/720/960 px) changes nothing. Also written: a backend implementation report.

4.7 Why the misses? Diagnostic and ground-truth check (7 Oct)

What we did. `diag_teachers.py` measured relevancy at three stages per object: lifting into 3D loses ~ 0.007 ; averaging the crop scales costs 0.07–0.09; several misses are CLIP recognition limits. Then every miss was re-examined in the photos.

Problem	Cause	Fix
purple foam mat “wrong object”	our annotation had leaked 2 m behind the mat; the model was right	re-placed from splat depth (amended set; frozen set kept)
glass door cabinet “wrong object”	the upper glass cabinets were not annotated as instances	instances added

backroom lift 8/13 $\rightarrow$ 10/13 on the amended set.

4.8 Teacher variants A, B, C (7 – 8 Oct)

What we did. Variant plumbing (`--clip-mode/--clip-model/--table-suffix`, tables record their CLIP model), then three branches: **A** multi-scale CLIP (three scale groups, chosen per query), **B** CLIP on SAM segments, **C** ViT-L/14. Each rebuilt on both scenes for lift, F-C and F-CD (long runs in tmux).

Problem	Cause	Fix
a running variant job's script was overwritten	bash reads a script while it runs; editing it changes what runs	original restored byte-identically; new scripts get new names (never edit a running script)
ViT-L/14 FMGS: tiny-cuda-nn out of memory	768-wide heads	fell back to PyTorch heads (slower)
B collapsed FMGS on flightroom (15/16 no candidate)	cells no SAM segment covers were 0; FMGS learned them	(B2, below)
merge conflicts merging main into B	both changed <code>teachers.py</code>	branches combined, both tests kept

Results (lift, /29): Old 20, **A 22**, B 19, C 18. A merged and made the default lift (8 Oct).

4.9 Improving B and C, and a floor sweep (8 Oct)

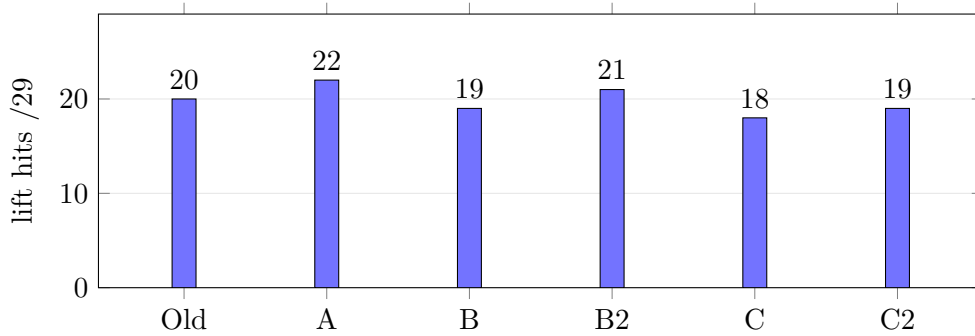


Figure 8: Lift (L-C) hits per teacher design at the default floor, amended query sets. With a floor sweep, B2 without level choice reaches 24/29 at the same false-positive count.

What we did. **B2:** pyramid fallback in uncovered cells, SAM segments split into part/object/region levels, mask cache. **C2:** ViT-L/14's floor calibrated by quantile mapping over a neutral vocabulary (0.5222). Then a floor sweep over all designs and an inspection of FMGS's misses. Results: B2 21, C2 19 at the default floor.

Finding	Evidence	Consequence
A’s lead is a floor effect	max over three groups raises every peak by ~ 0.045 (absent objects too); A without the choice = 20; Old at floor 0.51 = 23 with the same false positives	the default should be decided with a calibrated floor
best trade-off: B2 without level choice	24/29 with 2 of 6 absent objects firing	B2 flat is the strongest design measured
FMGS “no candidate” is not a floor problem	top-100 relevancies scattered 6–7.5 m apart, a third outside the camera volume	supervised-only baking and smoothing proposed
two “absent” objects fire everywhere	ceiling fan, recycling bin	the negative set needs a ground-truth check

Reports: `semantic_embedding_report.tex` and `semantics_primer.tex`.

5 Lessons

1. **Pin everything and never let an install move torch.** Most early failures were version drift (nerfstudio/gspat, pycolmap, OpenCV 5, numpy 2, timm). Every later install used `--no-deps` and a constraints file.
2. **Host problems look like software problems.** Kernel pinning, Secure Boot keys, suspend, RAM exhaustion during compiles and power cuts each cost hours; resumable steps with on-disk state turned interruptions into minor delays.
3. **Silent failures are the dangerous ones:** integer course cells, the frame sign convention, a wrong marker length, SfM folds, a stalled time optimiser reporting success. Each now has an explicit check.
4. **Measure before believing a limit.** The 6 GB training “minimum” was 2.4 GB with images on the CPU; the FMGS out-of-memory was allocator fragmentation, not size; A’s gain was mostly the floor.
5. **Ground truth needs checking too:** two of our own annotations were wrong, and two “absent” objects look present.
6. **Keep a development set apart from the evaluation set,** and freeze the evaluation set before results.

6 Timeline

Date	Step
25 Jul	Feasibility and first install on the 4 GB laptop; example flight runs
2 Aug	Clean rebuild with <code>install_figs.sh</code> ; four host faults fixed
6 Aug	First own capture-to-flight on the RTX 2080 (1 h 40 min)
7 Aug	<code>figs_pipeline.py</code> ; repository created
27 Sep	Galley phases 0–3 (job queue, splat page, course editor); SV-Net pipeline started
28 Sep – 1 Oct	intellisense08 bring-up; SV-Net gate (pipeline works, student does not fly); drone model; ribbon UI
3–4 Oct	splat view, keyboard flying, video upload, Google Drive import
5 Oct	Semantics phases 0–3 gated in one day; FMGS written
6 Oct	FMGS gate passed; Claude Code on the host; Phase 5 evaluation, power cuts, frozen sets, results
7 Oct	diagnostic, ground-truth check, variants A/B/C
8 Oct	A merged as default; B2, C2, floor sweep; reports